Almond FinBank Pro API - Senior Backend Interview Brief

Generated: 2026-07-27 17:37 UTC
Live API: https://almondsystems.com.ng/finbank-api/docs
Repository: https://github.com/kingkpono2/finbank-nestjs

Executive Summary

Almond FinBank Pro API is a NestJS/Node.js digital banking backend built to demonstrate senior backend engineering in a fintech context. It exposes REST APIs for authentication, account creation, transfers, payment webhooks, health checks, SMS simulation, and Swagger-based testing. The deployed stack uses PostgreSQL for durable financial records, Redis for account caching, RabbitMQ for asynchronous domain events, Brevo for transactional email, Docker Compose for service orchestration, and Nginx for production routing under /finbank-api.

The latest server hardening focused on production-style optimization: explicit PostgreSQL indexes, TypeORM auto-sync disabled for production, idempotent transfer retries, deterministic pessimistic locking, cache invalidation after balance changes, and verified Brevo email delivery for signup and transfer receipts.

What You Built

- You drove the FinBank product direction: a banking API that can be explained in a senior Node/NestJS fintech interview.
- You already had the application working in Swagger locally and wanted targeted improvements around Redis, RabbitMQ, email, SMS, deployment, and production readiness.
- You provided the repositories, server access, domain, product positioning, and Brevo credentials needed to make the app public and testable.
- You insisted on step-by-step debugging, which produced an interview-friendly understanding of the system instead of hidden changes.

What Codex Assisted With

- Deployed the FinBank API on the server using Docker Compose with PostgreSQL, Redis, RabbitMQ, and the NestJS API container.
- Configured Nginx routing so Swagger is available at https://almondsystems.com.ng/finbank-api/docs.
- Replaced fragile cache-manager usage with a direct Redis module/service pattern and fixed the missing await cache bug.
- Connected RabbitMQ microservice events for user registration and transfer completion.
- Troubleshot SMTP failures: confirmed Postfix accepted messages but outbound port 25 was blocked; confirmed Lark and Gmail auth failures; then integrated Brevo transactional email.
- Verified signup welcome email and transfer receipt email are recorded as SUCCESS using provider BREVO.
- Added Swagger JSON payload examples and fixed CSP so examples can render in Swagger UI.
- Added production optimization gaps: transfer idempotency, deterministic account locking, DB indexes, TypeORM production sync control, Redis cache invalidation, and safer cache reads.
- Pushed the Brevo email provider update to GitHub previously, and the current optimization changes should also be synced to Git after final verification.

Current Architecture

Client or Swagger sends REST requests to the NestJS API through Nginx. Auth requests issue JWT access and refresh tokens. Account APIs persist account records in PostgreSQL and use Redis for account-number cache reads. Transfer requests run inside an explicit PostgreSQL transaction using pessimistic row locks, debit/credit account updates, transaction record creation, and double-entry ledger rows. After commit, the service invalidates Redis account cache and publishes a RabbitMQ transfer.completed event. The event consumer sends transfer receipts through Brevo and logs mock SMS output. Audit logs capture the transfer action separately from the financial transaction.

Requirement Mapping

Node.js and NestJS: Implemented as a modular NestJS API with Auth, Users, Accounts, Transactions, Ledger, Events, Notifications, Redis, Audit, Health, Payments, Webhooks, and Mock/SMS modules.

RESTful APIs and microservices: REST endpoints serve mobile/web clients. RabbitMQ is used as the async microservice transport for domain events.

PostgreSQL expertise: Financial state is persisted in PostgreSQL. Transfers use database transactions, pessimistic locks, durable transaction rows, and ledger entries. Indexes were added for account, transaction, ledger, notification, and audit lookups.

Redis caching: Account-number lookup is cached with TTL, avoids caching nulls, and invalidates source/destination account cache entries after a successful transfer.

High transaction volume and data integrity: The transfer path uses row-level locks, deterministic lock ordering, transaction rollback on errors, double-entry ledger rows, and idempotency keys to prevent duplicate debit on retry.

Third-party integrations: Mock payment gateway and webhook endpoint exist for interview/demo purposes. Brevo transactional email is integrated for real outbound signup and transfer emails.

Security: JWT auth protects account and transfer APIs. Helmet, validation whitelist, non-whitelisted request rejection, refresh token handling, and audit logs are present. Secrets are kept in .env, not source control.

Docker/cloud: The app is deployed on a cloud VPS using Docker Compose and Nginx reverse proxy.

Optimizations Implemented

1. TypeORM synchronize is now controlled by TYPEORM_SYNC and set false in production. This avoids accidental schema drift in live financial systems.
2. Explicit indexes were added:
   - transaction fromAccount + createdAt
   - transaction toAccount + createdAt
   - transaction status + createdAt
   - unique idempotency key index
   - account owner + status
   - ledger account + createdAt
   - ledger reference
   - audit user/action + createdAt
   - notification recipient/status/provider indexes
3. Transfer idempotency was added using client-supplied idempotencyKey. Retrying the same transfer returns the original reference with idempotentReplay true.
4. Transfer locking now locks both account rows in deterministic account-number order to reduce deadlock risk under concurrent opposite-direction transfers.
5. Redis cache safety was improved: account lookups only cache real account rows and transfer success invalidates both affected account cache keys.
6. Swagger CSP was adjusted so inline example population can work in the hosted docs page.
7. Brevo transactional email avoids VPS port 25 restrictions and sends through a legitimate provider.

Verified Production Tests

Health check: GET /finbank-api/health returned status UP.

Registration: POST /finbank-api/auth/register returned 201 Created and produced a notification_log row with provider BREVO and status SUCCESS.

Transfer with retry: A live transfer with idempotencyKey interview-transfer-1785173689614 returned success. Retrying the same request returned the same reference and idempotentReplay true.

Database integrity: The idempotency key had exactly one transaction row. The transfer reference had exactly two ledger entries, one debit and one credit.

Email: Transfer receipt for reference TXN-1785173690778-2902 was saved as SUCCESS / BREVO.

Swagger: Public docs JSON includes idempotencyKey and the content security policy now allows the Swagger example filler script.

Swagger Test Payloads

Register:
POST /finbank-api/auth/register
```
{
  "firstName": "Kpono-Abasi",
  "lastName": "Akpabio",
  "email": "change-me+001@gmail.com",
  "phone": "+2348011112201",
  "password": "Password1"
}
```

Login:
POST /finbank-api/auth/login
```
{
  "email": "change-me+001@gmail.com",
  "password": "Password1"
}
```

Create account:
POST /finbank-api/accounts
Authorization: Bearer <accessToken>
```
{
  "type": "SAVINGS",
  "initialBalance": 50000
}
```

Transfer:
POST /finbank-api/transactions/transfer
Authorization: Bearer <accessToken>
```
{
  "fromAccount": "replace-with-source-accountNumber",
  "toAccount": "replace-with-destination-accountNumber",
  "amount": 2500,
  "narration": "Live Swagger demo transfer",
  "idempotencyKey": "demo-transfer-20260727-001"
}
```

SMS mock:
POST /finbank-api/mock/sms
```
{
```

```
  "phone": "+2348011112299",
  "message": "Your Almond FinBank demo transfer of NGN 2500 was successful."
}
```

Webhook mock:

```
POST /finbank-api/webhooks/payment
{
  "provider": "flutterwave",
  "reference": "FLW-DEMO-20260727-001",
  "accountNumber": "replace-with-accountNumber",
  "amount": 25000,
  "currency": "NGN",
  "status": "SUCCESS",
  "paidAt": "2026-07-27T12:00:00.000Z"
}
```

Likely Interview Questions and Strong Answers

Q: Walk me through the architecture.
A: It is a modular NestJS banking backend. REST controllers handle auth, accounts, transfers, mocks, webhooks, and health. PostgreSQL is the source of truth. Redis is used for account lookup caching. RabbitMQ decouples side effects like emails and SMS from the core transfer transaction. Brevo sends transactional email. Docker Compose runs API, Postgres, Redis, and RabbitMQ, while Nginx exposes the API under /finbank-api.

Q: How do you ensure transfer integrity?
A: Transfers run in an explicit database transaction. The service locks the two account rows with pessimistic write locks, validates status and balance, updates balances, creates one transaction record, and writes debit and credit ledger entries. If any step fails, it rolls back. Side effects like email are published only after commit.

Q: Why add idempotency?
A: In banking, clients retry requests because of network timeouts. Without idempotency, a retry could debit twice. The API now accepts an idempotencyKey, stores it uniquely, and returns the original transaction reference on replay.

Q: How did you reduce deadlock risk?
A: Instead of locking sender then receiver in request order, the service locks both accounts in deterministic account-number order. This avoids two concurrent opposite-direction transfers each holding one lock and waiting on the other.

Q: What did Redis do here?
A: Redis caches account lookups by account number with a short TTL. After a transfer, both affected account cache keys are invalidated so balance reads do not serve stale state.

Q: Why RabbitMQ?
A: Email and SMS are non-critical side effects. RabbitMQ lets the transfer commit quickly and reliably, then the consumer processes notifications asynchronously. It improves resilience and avoids coupling banking consistency to an email provider outage.

Q: What indexes did you add and why?
A: I added indexes for transaction history by source/destination account and date, status/date for operational queries, ledger account/date and reference lookup, audit user/action/date, notification recipient/status/provider, and a unique idempotency key. These match common financial query paths.

Q: Why disable TypeORM synchronize in production?
A: Auto-sync is convenient in development but risky in production because schema changes can happen implicitly at startup. Financial systems should use explicit migrations or controlled SQL scripts.

Q: How do you handle email delivery?
A: Direct Postfix was blocked on outbound port 25 and would also have SPF issues. I integrated Brevo transactional email via API, configured sender identity, and verified successful signup and transfer receipt logs.

Q: What would you improve next?
A: Add proper TypeORM migrations, CI/CD, automated e2e tests, structured logging, metrics, tracing, a real SMS provider, webhook signature verification, rate limiting, RBAC policies, reconciliation jobs, settlement flows, and a secrets manager.

Remaining Production Roadmap

- Replace demo SQL patching with versioned TypeORM migrations.
- Add CI pipeline for lint, unit tests, e2e tests, and Docker build.
- Add Prometheus/OpenTelemetry metrics and centralized logs.
- Add request correlation IDs across REST, RabbitMQ, and notification logs.
- Add webhook signature verification and replay protection.
- Add real SMS provider integration.
- Add admin/support authorization policies.
- Add reconciliation and settlement modules for complete banking operations.
- Move secrets to a cloud secrets manager.

Interview Positioning

Position this app as a practical fintech backend demo that you hardened iteratively: you started with working Swagger APIs, then added production concerns around data integrity, caching, messaging, email delivery, deployment, and optimization. Be honest that it is not a full bank core, but it demonstrates the exact senior backend patterns needed in regulated financial systems: transactional correctness, retries, idempotency, async side effects, auditability, deployment, and performance-aware database design.